

Operators in SQL

SQL operators are special keywords or characters used in SQL statements to perform operations such as comparisons, arithmetic calculations, and logical operations. These operators are crucial for constructing complex queries and manipulating data effectively.

Types of Operators

SQL operators can be broadly categorized into several types:

Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric values.

Operator	Description	Example
+	Addition	<code>SELECT 10 + 5;</code> (Result: 15)
-	Subtraction	<code>SELECT 10 - 5;</code> (Result: 5)
*	Multiplication	<code>SELECT 10 * 5;</code> (Result: 50)
/	Division	<code>SELECT 10 / 5;</code> (Result: 2)
%	Modulo (remainder)	<code>SELECT 10 % 3;</code> (Result: 1)

Comparison Operators

Comparison operators are used to compare two expressions and return a boolean value (true, false, or unknown).

Operator	Description	Example
=	Equal to	<code>SELECT * FROM Products WHERE Price = 20;</code>
!= or <>	Not equal to	<code>SELECT * FROM Products WHERE Price != 20;</code>

Operator	Description	Example
>	Greater than	<code>SELECT * FROM Products WHERE Price > 20;</code>
<	Less than	<code>SELECT * FROM Products WHERE Price < 20;</code>
>=	Greater than or equal to	<code>SELECT * FROM Products WHERE Price >= 20;</code>
<=	Less than or equal to	<code>SELECT * FROM Products WHERE Price <= 20;</code>

Logical Operators

Logical operators combine multiple conditions to create more complex expressions.

Operator	Description	Example
AND	Returns true if all conditions are true	<code>SELECT * FROM Orders WHERE Quantity > 10 AND OrderDate > '2023-01-01';</code>
OR	Returns true if any condition is true	<code>SELECT * FROM Customers WHERE City = 'New York' OR City = 'London';</code>
NOT	Reverses the boolean value of the condition	<code>SELECT * FROM Employees WHERE NOT Department = 'HR';</code>
BETWEEN	Checks if a value is within a range	<code>SELECT * FROM Products WHERE Price BETWEEN 10 AND 50;</code>
IN	Checks if a value matches any value in a list	<code>SELECT * FROM Customers WHERE Country IN ('USA', 'Canada', 'Mexico');</code>

Operator	Description	Example
LIKE	Searches for a specified pattern in a column	<code>SELECT * FROM Products WHERE ProductName LIKE 'A%';</code>
IS NULL	Checks for NULL values	<code>SELECT * FROM Employees WHERE Email IS NULL;</code>
IS NOT NULL	Checks for non-NULL values	<code>SELECT * FROM Employees WHERE Email IS NOT NULL;</code>

Bitwise Operators

Bitwise operators perform manipulations at the bit level of integer values.

Operator	Description	Example
&	Bitwise AND	<code>SELECT 5 & 3; (Result: 1)</code>
	Bitwise OR	<code>SELECT 5 3; (Result: 7)</code>
^	Bitwise XOR	<code>SELECT 5 ^ 3; (Result: 6)</code>

Operator Precedence

Operator precedence determines the order in which operators are evaluated in an expression. Operators with higher precedence are evaluated before operators with lower precedence. Parentheses () can be used to override the default precedence.

For example, in the expression `SELECT 5 + 2 * 3;`, the multiplication (*) is performed before the addition (+), resulting in `5 + 6 = 11`. If you want the addition to be performed first, you would use `SELECT (5 + 2) * 3;`, which results in `7 * 3 = 21`.